APPLIED MATH 3

WEEK 5

OBJECT HIERARCHIES

REVIEW

CLASS HIERARCHIES

- The concept of a Class Hierarchy or (also called Object Hierarchies or Inheritance) is an important idea in computer science.
- It is particularly important for our work with Actors, Pawns and Characters in Unreal as those are actually all related through their hierarchy.
- Before we can tackle this however, we will talk about class Hierarchies more broadly.

CLASS HIERARCHIES

- The basic idea of a class hierarchy is that one class can Inherit from another class.
- Parent class: the class that is being inherited from by the child.
- Child class: the class that is inheriting from the parent.
- What this means is that the Child class is considered to be an Object of the Parent object type. It also has all the variables and methods that the parent class has.
- The child class can then add additional variables and methods, or even override the parent class methods, by defining a new version of the same method.

CLASS HIERARCHIES

- Consider a simple example:
- We want to define two object types, one to represent a Dog, and another object type to represent a Cat.
- Each class will have the following variables:
  - integer: numberOfLegs
  - String: soundMade

CLASS HIERARCHIES

```
Dog{
    int numOfLegs = 4;
    String sound = “Bark”;
}

Cat{
    int numOfLegs = 4;
    String sound = “Meow”;
}
```

CLASS HIERARCHIES

Adding some methods:

```
Dog{
    void MakeSound()
    void DigUpYard()
}

Cat{
    void MakeSound()
    void MakeHairball()
}
```

CLASS HIERARCHIES

Now we have some of the same code in both classes, however, each class has some functions that the other shouldn't have.

This may be a good time to implement a Class Hierarchy to help reduce, reuse and recycle our old code.

We could create a new class, called Animal, that we could make the parent class of both Dog and Cat.

CLASS HIERARCHIES

Parent Class

```
class Animal{
    Int numOfLegs = 4;
    String sound =
    “Undefined”;
    void MakeSound()
}
```

Child Classes

```
class Dog:Animal{
    void DigUpYard()
}

class Cat:Animal{
    void MakeHairball()
}
```

CLASS HIERARCHIES

- Dog is a child class of Animal
- : or :: means Dog extends the Animal class (other languages may use different syntax but the concept is widely used).
- This means our Dog counts as both a Dog object and an Animal object.
- This also means that every Cat and Dog will automatically have all of the methods and variables defined in Animal, without having to explicitly declare them.
- This also means we could cast the Dog object to an Animal variable, and call any methods that Animal contains.

```
class Dog : Animal {.. }
```

CLASS HIERARCHIES

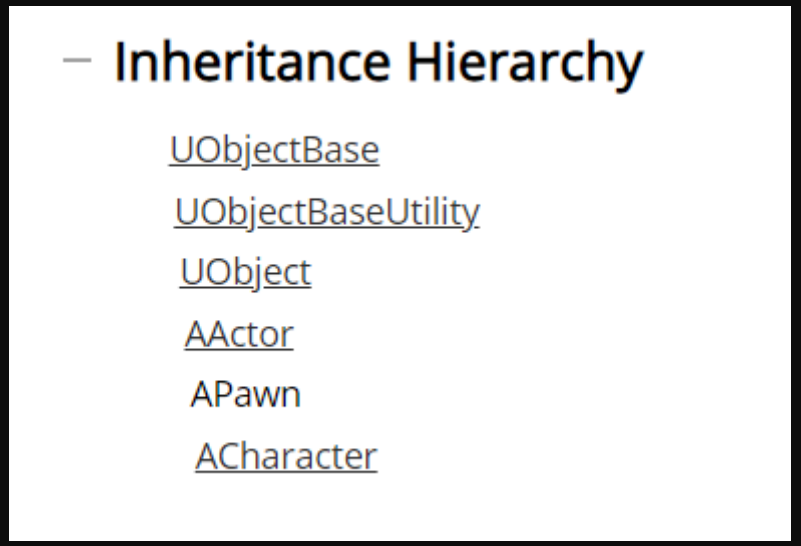
- This may not seem powerful at first glance but it is extremely useful.
- A simple advantage is apparent if we wanted to do some operation on all Cats and all Dogs. Without subclasses, we would have to create two separate lists, one for each of Cat and Dog objects.
- With subclasses we could create a single list of type Animal, and place all Cats and all Dogs in that list.
- We could then call any Animal method on any object in the list.

CLASS HIERARCHIES

- Another useful scenario with inheritance is in allowing you to add special behaviour to a class through subtyping.
- This is very common in unreal. When we create a blueprint of type Pawn (call it MyPawn), what we are really doing is creating a new subclass of type Pawn.
- What this means is our new custom class MyPawn contains all of the variables and methods that Pawn has. This is where our built in methods interact with our code.

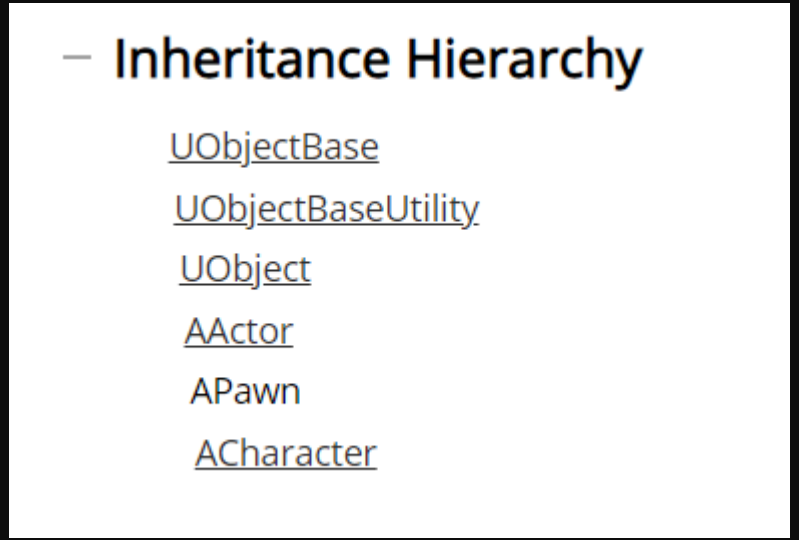
CLASS HIERARCHIES

- If you go to the Unreal documentation, you can see the object hierarchy listed at the top of the page
- (Inheritance Hierarchy) https://docs.unrealengine.com/4.27/en-US/API/Runtime/Engine/GameFramework/APawn/



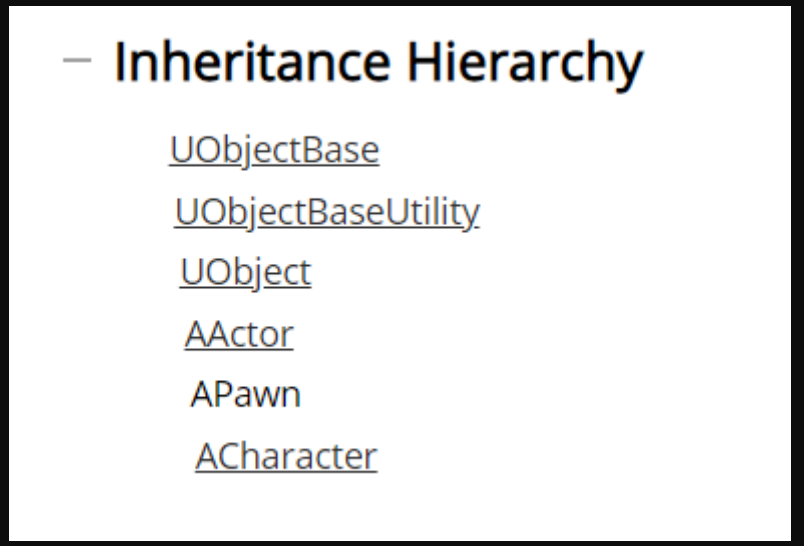
CLASS HIERARCHIES

- A note on the U and A prefixes:
- In Unreal, classes that extend from UObject get an automatic U added in front of the name.
- You don't have to add this prefix if you are creating the C++ class through UE.
- Classes that extend from an Actor class (AActor) get an A added.
- When I am referring to the Pawn class, or the Actor class, I am actually talking about APawn and AActor, it just becomes cumbersome to say each time. They are the same class though.
- There are several others, you can see the full list at the link below.
- https://docs.unrealengine.com/5.0/en-US/epic-cplusplus-coding-standardblueprint-debugging-in-unreal-engine/



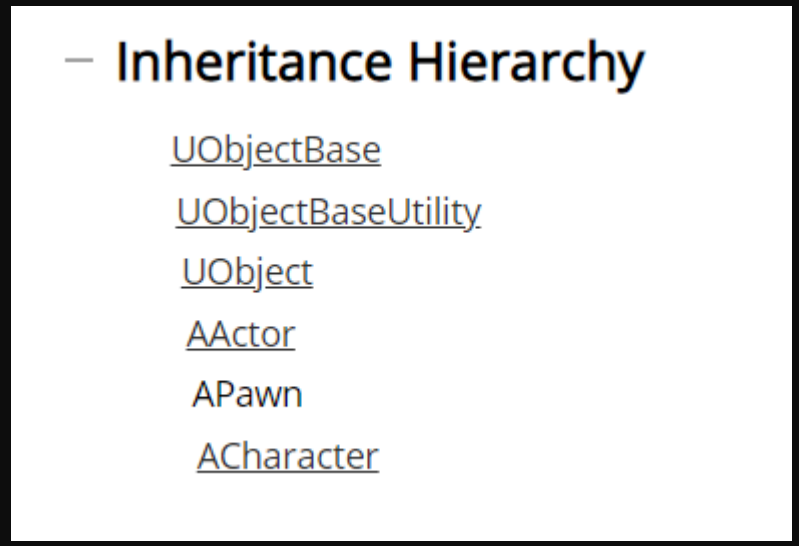
CLASS HIERARCHIES

- What this diagram is telling us is that APawn is a child object of AActor, which is a child of UObject, which is a child of UObjectBaseUtility, which is itself a child of UObjectBase.
- We can also see that Character is a subclass of Pawn.
- This should make some sense from our earlier introduction to these object types.
- If you go to the link for UObject, you will see that it is the base class of all Unreal Engine objects.
- UObjectBase and UObjectBaseUtility are internal and should not be used directly.
- If you want a simple object, use UObject



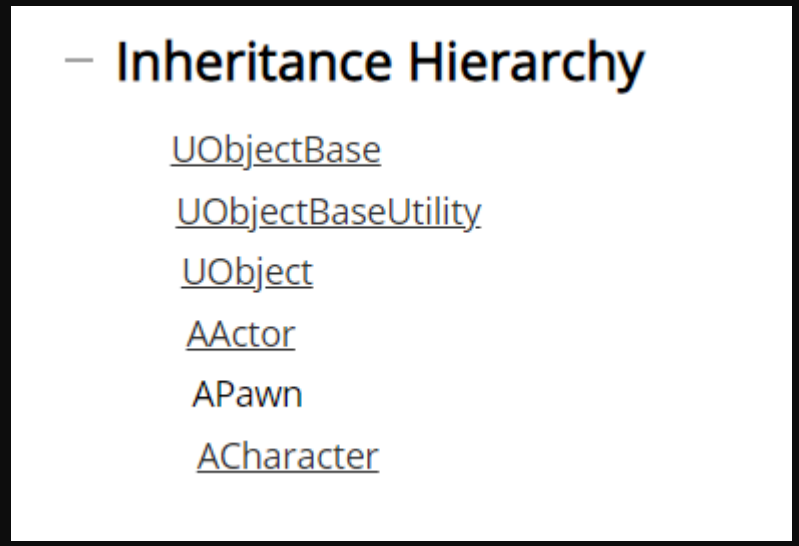
CLASS HIERARCHIES

- Thinking about it another way is that the Object class creates the functionality for all basic objects, whether they are in the scene or not.
- When the AActor class extends UObject, it adds the functionality that allows that UObject to exist in the game world. Position, rotation and other related data would be included at this level.
- When Pawn extends Actor, we are getting the functionality for connecting with a Controller to allow for AI or Player control.
- Character is just a Pawn that has extra logic for movement, jumping and other actions added to it.



CLASS HIERARCHIES

- We haven't talked about Controllers in great detail yet but they are another example of a class Hierarchy in Unreal.
- Controller is the parent class for both AIController and PlayerController.
- This means that the Pawn class is designed to work with the Controller class. Both the AIController and PlayerController are themselves Controller classes, therefore the Pawn can be controlled by either an AIController or a PlayerController the same way.
- This is a specific design decision made by the Unreal programmers but the result is that AI and Players are easily interchangeable, since they both interact with the Pawn the same way.



CLASS HIERARCHIES - ACCESS

- You have probably encountered the variable access modifiers “public” and “private”,
  - Public: accessible to any other object
  - Private: not accessible to any other object

CLASS HIERARCHIES - ACCESS

- You have probably encountered the variable access modifiers “public” and “private”,
- The introduction of subclasses also brings with it a new access modifier: Protected.
- A method or variable that is protected will be accessible only to the object itself, or to any child class of the object.

CLASS HIERARCHIES - ACCESS

- To demonstrate this, imagine a simple class:

```
class Person{
    public int age;
    private int idNumber;
    protected string name;
}
```

CLASS HIERARCHIES - ACCESS

If you created an object of type Person

Any object with access to the p object could access the age, however it would not be able to access either the idNumber or the name.

```
class Person{
    public int age;
    private int idNumber;
    protected string name;
}
```

Person p = new Person();

p.name X

p.age OK

p.idNumber X

CLASS HIERARCHIES - ACCESS

- If we then created a subclass of Person called Student.
- Our student object would have access to the age and name, but would not have (direct) access to the idNumber.
- The Student would still HAVE an idNumber variable, but could only access it through (public or protected) methods defined in Person.

```
class Person{
        public int age;
        private int idNumber;
        protected string name;
}
class Student:Person{
        void doStuff(){
                age = 20; // OK
                idNumber = 111; // ERROR
                name = "New Name"; // OK
        }
}
```

CLASS HIERARCHIES - CASTING

- Another important concept is the idea of "Casting" data to a different datatype.
- When we cast a variable, we store an object in a variable of its parent type.

```
class Person{
    public int age;
    private int idNumber;
    protected string name;
}

class Student:Person{
    …
}
```

CLASS HIERARCHIES - CASTING

- In our current example, we could “cast” our Student to a Person variable.
- The Student object data will still be stored in memory, however it will not be accessible until it is “cast” back into a Student variable.

```
class Person{
    public int age;
    private int idNumber;
    protected string name;
}

class Student:Person{
    …
}
```

CLASS HIERARCHIES - CASTING

```
// Regular Person object
Person p = new Person();
// Regular Student object
Student s = new Student();
```

```
class Person{
    public int age;
    private int idNumber;
    protected string name;
}

class Student:Person{
    …
}
```

CLASS HIERARCHIES - CASTING

- C++ Dynamic Casting:
  - dynamic_cast < type-id > (expression)

```
Student s = new Student();
Person* p = dynamic_cast<Person>(s);
//p is now considered a Person object until it is cast back to Student.
Student* s = dynamic_cast<Student>(p);
// s is now considered a Student again, and all data and methods are accessible
```

- https://docs.microsoft.com/en-us/cpp/cpp/casting

CLASS HIERARCHIES - CASTING

```
// Student cast to Person with
Student s = new Student();
Person* sp = dynamic_;
// valid cast
sp.personMethod();
// invalid, considered a Person
type object and so doesn't
recognize the Student methods.
sp.studentMethod();
```

```
class Person{
    public int age;
    private int idNumber;
    protected string name;
}

class Student:Person{
    …
}
```

CLASS HIERARCHIES - CASTING

- This is really getting too deep for this class and will be discussed in greater detail in Programming 2 and 3.
- For more information see: https://docs.microsoft.com/en-us/cpp/cpp/casting

CLASS HIERARCHIES – POLYMORPHISM

Where this starts to become really powerful is with a new concept called Polymorphism.

Students tend to get this mixed up with the more general idea so make sure you have a good solid understand of subclasses up to this point before proceeding.

CLASS HIERARCHIES – POLYMORPHISM

- When making a subclass of an object, we have access to the parent objects protected and public methods.
- We can also extend that functionality by overriding parent class methods.
- To override a method, we provide the subclass with the exact same method definition as the child class.
- If you have an object of the child type, stored in a variable of the parent type, and the child type has overridden a method in the parent, when that method is called on the object, the childs version of the method will be dynamically called instead.
- This is a little confusing so lets revisit our Animal example from earlier.

CLASS HIERARCHIES

Parent Class

```
class Animal{
        Int numOfLegs = 4;
        String sound = "Undefined";
        void MakeSound()
        // new method
        void GetSpecies(){
                cout <<  "Animal";
        }
}
```

- Both child classes Dog and Cat would be able to call the GetSpecies() method, however they would both also print "Animal", which isn't ideal.

```
class Dog:Animal{…}

class Cat:Animal{…}

Cat c = new Cat();
c.GetSpecies(); // "Animal"
Dog d = new Dog();
d.GetSpecies(); // "Animal"
```

CLASS HIERARCHIES

Parent Class

```
class Animal{
        Int numOfLegs = 4;
        String sound = "Undefined";
        void MakeSound()
        // new method
        void GetSpecies(){
                cout <<  "Animal";
        }
}
```

- By overriding the methods in the child classes, we can provide better output.

```
class Dog:Animal{
        void GetSpecies(){
                cout << "Dog";
        }
}

Dog d = new Dog();
d.GetSpecies(); // "Dog"
```

CLASS HIERARCHIES

Parent Class

```
class Animal{
    Int numOfLegs = 4;
    String sound = "Undefined";
    void MakeSound()
    // new method
    void GetSpecies(){
        cout <<  "Animal";
    }
}
```

- By overriding the methods in the child classes, we can provide better output.

```
class Cat:Animal{
    void GetSpecies(){
        cout << "Cat";
    }
}

Cat d = new Cat();
d.GetSpecies(); // "Cat"
```

CLASS HIERARCHIES

Parent Class

```
class Animal{
        Int numOfLegs = 4;
        String sound = "Undefined";
        void MakeSound()
        // new method
        void GetSpecies(){
                cout <<  "Animal";
        }
}
```

- Note: even though the methods are overridden in the child classes (Cat and Dog), the original method still persists if you had an Animal class itself.
- Animal a = new Animal();
- a.GetSpecies(); // "Animal"

THIS IS THE POLYMORPHISM PART!

```
class Animal{
        Int numOfLegs = 4;
        String sound = "Undefined";
        void MakeSound()
        // new method
        void GetSpecies(){
                cout <<  "Animal";
        }
}
class Cat:Animal{
        void GetSpecies(){
                cout << "Cat";
        }
}
```

- Polymorphism occurs when you have a child class object stored in a parent variable, then call a method overwritten in the parent.
- The object will dynamically select the child class method if it is available.
- Animal* a = new Cat();
- a.GetSpecies(); // polymorphism selects the Cat version of the GetSpecies method, our output is "Cat"

THIS IS THE POLYMORPHISM PART!

```
class Animal{
        Int numOfLegs = 4;
        String sound = "Undefined";
        void MakeSound()
        // new method
        void GetSpecies(){
                cout <<  "Animal";
        }
}
class Cat:Animal{
        void GetSpecies(){
                cout << "Cat";
        }
}
```

- Polymorphism allows us to have different types of objects all get called together, with different results!

```
// Pseudo code
ListOfAnimals list = new List;
list.add(new Animal());
list.add(new Cat());
list.add(new Dog());
// iterate list and call GetSpecies on all objects:
"Animal", "Cat", "Dog" is returned.
```

THIS IS THE POLYMORPHISM PART!

```
class Animal{
        Int numOfLegs = 4;
        String sound = "Undefined";
        void MakeSound()
        // new method
        void GetSpecies(){
                cout <<  "Animal";
        }
}
class Cat:Animal{
        void GetSpecies(){
                cout << "Cat";
        }
}
```

- Polymorphism allows us to have different types of objects all get called together, with different results!

```
// Pseudo code
ListOfAnimals list = new List;
list.add(new Animal());
list.add(new Cat());
list.add(new Dog());
// iterate list and call GetSpecies on all objects:
"Animal", "Cat", "Dog" is returned.
```

OK SO WHAT?

- So why have we learned all this?
- Well, in Unreal, we are dealing several hierarchies.
- Some of the examples we have looked at so far include:
  - Object > Actor > Pawn > Character
  - Controller > AIController
  - Controller > PlayerController
- It is important to understand this relationship so that we understand which methods are available to us.
- If we create a custom Pawn, we can access any method from either Pawn or Actor.